

C1 — Test Strategy, Risk & Metrics

Service: Transactions Service

Scope:

1. POST /accounts
2. GET /accounts/{accountId}
3. POST /transactions

Based On: OpenAPI contract (contract/transactions-service.v1.yaml) and contract-audit findings (artificats/C3a_contract-findings.pdf).

1. Risk Model

Severity:

- **P0** means a failure could corrupt financial data, move money twice, or break audit records.
- **P1** means significant contract/functional risk, release-blocking when exercised by a consumer or critical flow.
- **P2** is an important quality issue, but not a reason to stop a money-moving release.

This follows the contract-audit severity model (docs/[C3a-contract-audit.md](#)).

Rank	Failure mode	Severity	Business impact
R1	Duplicate transaction during retry/concurrency.	P0	Double debit/credit cause direct financial loss and make it difficult to reconcile the accounts.
R2	Incorrect transaction sign/type	P0	Balance changes in the wrong direction, which could cause widespread financial errors.
R3	Incorrect amount/currency/precision handling	P0	Amount recorded incorrectly, causing customer balances and financial records to be inaccurate.
R4	Business write succeeds but audit write does not (or vice versa)	P0	Financial records cannot be reconciled reliably.
R5	Same idempotency key reused with a different payload	P0	A retry key can become associated with two different financial operations.
R6	Transaction created for an invalid/non-existent account	P0	Money could be linked to the wrong account or left without a valid account.
R7	Unsupported operation type reaches a financial transaction	P0	An invalid operation can change money in an incorrect way.
R8	Lost transaction	P0	The customer believes money moved but the system of record does not

			contain it.
R9	Account identity duplicated	P0	Transactions attached to the wrong account.
R10	Missing/incorrect request correlation ID	P1	Slower incident investigation and weak traceability.
R11	Unexpected schema/status response	P1	Consumers could break or handle the response incorrectly.
R12	Slow endpoint (increased latency)	P2 initially, but P1 if SLA breach affects money flows	Slow APIs can affect customers and may trigger retries, which can increase the risk of duplicate transactions.
R13	Malformed or oversized requests	P2	Bad or very large requests can cause reliability, security, or resource-usage problems.

2. Test Approach and Pyramid

2.1 Test Layers

Layer	What belongs here?	Example	Owner
Unit	Pure business rules	Amount sign changes, amount conversion/rounding, input validation, duplicate-request detection, error mapping	Dev
Contract	Tests to verify that changes to an API don't unexpectedly break the services or clients that depend on it.	Required fields, data types, allowed values, headers, status codes, response schema, and request/response compatibility	Shared – Dev owns the provider contract and QA independently validates the API against it.
Functional / API	Business behavior that can be observed through the API	Account creation, transaction correctness, invalid accounts, invalid operations, amount limits, and duplicate-request handling.	QA + Dev
Integration	How different parts of the system work together and whether data stays correct	DB constraints, transactions atomicity, audit records, consistent business data, and saved state	Dev owns implementation tests; QA focuses on risk scenarios and integration validation
E2E	Few critical user journeys from start	Create an account → perform different types of transactions → verify the	QA

	to finish.	transaction records	
Production Observability	Signals that show if the system is working correctly.	Balance mismatches, duplicate transactions, error rates, latency SLAs, and audit failures	Shared – Dev, QA, SRE/DevOps

2.2 Where contract testing earns its keep?

Consumer testing is the cheapest way to catch provider/consumer incompatibility before expensive functional or E2E test runs.

For this service, contract validation should cover:

1. request/response schema
2. required fields
3. field types and formats
4. operation/status mappings
5. documented response headers
6. backward compatibility of existing fields/statuses

Since the current contract is under specified, contract testing cannot validate a business rule the schema doesn't define. Therefore, I would fail the contract quality gate for a money-moving release until the contract gaps are fixed.

2.3 What I deliberately would not test?

1. **Internal implementation details:** private methods or specific database queries. These are better covered by developer-owned unit and integration tests.
2. **Every permutation at E2E:** Most combinations should be covered at the unit and API levels. E2E tests should focus on a few important business flows.
3. **UI Behaviour:** The transactions service only exposes APIs, so UI testing is not applicable here.
4. **Performance claims from Prism:** Prism is a contract-based mock. Performance testing should be done in a production-like staging environment.
5. **Unspecified business rules as assertions:** I won't assume a document number length or amount decimals precision unless the contract defines it.

3. Quality Metrics

3.1 Metrics for Release Manager

Metric	What does it tell?	Release Signal
Critical money flow correctness	Did purchases, withdrawals, and vouchers result in the correct financial state?	Zero unresolved P0 failures
Duplicate transaction rate	Are retries creating duplicate transactions?	Zero unexplained duplicates

Reconciliation mismatch rate	Do business records match the audit records?	Zero unexplained mismatches
Idempotency pass rate	Do multiple requests with the same key create exactly one transaction?	100% pass rate before release
Contract-breaking changes	Did the provider break a consumer contract?	Zero unapproved breaking changes
Open P0/P1 defects	What serious known issues are still open?	P0 = 0, P1 needs explicit approval.
Escaped defect rate	Are bugs reaching staging or production that should have been caught earlier?	Should trend downward; investigate any increase
Test flake rate	Can the test suite be trusted?	Keep it low
API error rate	Is the service handling requests reliably?	Within agreed SLOs.
p95/p99 latency	Is performance within the agreed SLO?	Within agreed SLOs.

3.2 Vanity Metrics

1. **Pass Rate:** A 99% pass rate can hide one failed P0 test among thousands of low-value tests. It should be replaced with a quality gate holding tests validating business critical use cases with 100% pass rate.
2. **Test Automation Percentage:** More tests do not mean more risk is retired. It should be replaced with risk coverage – measuring P0 and P1 tests with automated coverage.
3. **Code coverage %:** High coverage does not mean that important financial rules are properly tested. Replace with: Checking if critical business rules are tested, especially invalid scenarios and cases designed to catch incorrect behavior.

I would collect these metrics for engineering health, but I would not use them as release evidence.

3.3 Dashboard Sketch

Transactions Service - Release Quality

Release Gate			
- Contract breaking changes	0		[Pass]
- Open P0 defects	0		[Pass]
- Critical money flow tests	100%		[Pass]
- Idempotency exactly one transaction	100%		[Pass]
- Reconciliation mismatch	0		[Pass]
- Open P1 defects	1		[Approval Required]
Automation Trust			
- Risk Coverage [P0/P1]	95%		

- Flaky Test Rate	1%
- Defect Escape Rate	Downward trend
- Last Green Run	<timestamp>
- Deployment Tracking	No prod deployment skipped test env.
Performance Health	
- p95 latency	150ms
- p99 latency	300ms
- 5xx error rate	0.1%

Note: The exact latency/error thresholds should come from agreed SLOs.

4. Open Questions

4.1 Release blocking questions

Question	Why does it matter?
What is the API currency and amount precision?	Without this information, we can't clearly determine whether an amount is correct.
What are the exact valid amounts: minimum, maximum, and zero/negative behavior?	Without these rules, incorrect amounts could lead to financial errors.
What is the exact sign/type mapping for operation IDs 1–4?	Incorrect signs can change the financial state in the wrong direction.
What is idempotency key header, and format?	Without an idempotency key, we can't be sure that retries won't create duplicate transactions.
What happens for the same idempotency key + same body, and same key + different body?	Without clear rules, retries could create duplicate or conflicting transactions.
How can QA prove exactly one transaction was created after concurrent replay?	Current APIs don't provide a way to check the final transaction count.
How is audit creation verified, and is business + audit write atomic?	We need to prove that the transaction and its audit record is consistent.
What is the “document_number” uniqueness rule?	If it isn't unique, transactions can link to the wrong account.
What are the authentication/authorization requirements?	Consumers cannot determine or test who may mutate financial state.

4.2 Non-blocking questions

Question	Why does it matter?
Should unknown JSON properties be rejected or ignored?	Important for compatibility behaviour.

Should “event_date” use ISO date-time schema format present in the contract example?	Makes the API easier to validate and use consistently across different systems.
What is the exact request-ID header format?	Required for observability and transaction tracing across services.
What are the agreed p95/p99 latency and error-rate SLOs?	Needed before adding performance to release criteria.

Note: Performance latency question is non-blocking for functional validation, but blocking for a performance release gate.